Parte I

Punto de partida

Capítulo 1

¿Qué es un sistema operativo?

El *sistema operativo* es el principal programa que se ejecuta en toda computadora de propósito general.

Los hay de todo tipo, desde muy simples hasta terriblemente complejos, y entre más casos de uso hay para el cómputo en la vida diaria, más variedad habrá en ellos.

A lo largo del presente texto, no se hace referencia al sistema operativo como lo *ve* o *usa* el usuario final, o como lo vende la mercadotecnia — el ambiente gráfico, los programas que se ejecutan en éste, los lenguajes de programación en los cuales están desarrollados y en que más fácilmente se puede desarrollar para ellos, e incluso el conjunto básico de funciones que las bibliotecas base ofrecen son principalmente *clientes* del sistema operativo — se ejecutan sobre él, y ofrecen sus interfaces a los usuarios (incluidos, claro, los desarrolladores). La diferencia en el uso son sólo –cuando mucho– *consecuencias* del diseño de un sistema operativo. Más aún, con el mismo sistema operativo –como pueden constatarlo comparando dos distribuciones de Linux, o incluso la forma de trabajo de dos usuarios en la misma computadora– es posible tener *entornos operativos* completamente disímiles.

1.1. ¿Por qué estudiar los sistemas operativos?

La importancia de estudiar este tema radica no sólo en comprender los mecanismos que emplean los sistemas operativos para cumplir sus tareas sino en entenderlos para evitar los errores más comunes al programar, que pueden resultar desde un rendimiento deficiente hasta pérdida de información.

Como desarrolladores, comprender el funcionamiento básico de los sistemas operativos y las principales alternativas que ofrecen en muchos de sus puntos, o saber diseñar algoritmos y procesos que se ajusten mejor al sistema operativo en que vayan a ejecutarse, puede resultar en una diferencia cualitativa decisiva en el producto final.

Parte de las tareas diarias de los administradores de sistemas incluye enfrentarse a situaciones de bajo rendimiento, de conflictos entre aplicaciones, demoras en la ejecución, y otras similares. Para ello, resulta fundamental comprender lo que ocurre *tras bambalinas*. Los sistemas de archivos resultan un área de especial

interés para administradores de sistemas: ¿cómo comparar las virtudes y desventajas de tantos sistemas existentes, por qué puede resultar conveniente mezclar distintos sistemas en el mismo servidor, cómo evitar la corrupción o pérdida de información? Lo que es más, ¿cómo recuperar información de un disco dañado?

En el área de la seguridad informática, la relación resulta obvia. Desde el punto de vista del atacante, si le interesa localizar vulnerabilidades que permitan elevar su nivel de privilegios, ¿cómo podría lograrlo sin comprender cómo se engranan los diversos componentes de un sistema? La cantidad de tareas que debe cubrir un sistema operativo es tremenda, y se verán ejemplos de sitios donde dicho atacante puede enfocar sus energías. Del mismo modo, para quien busca *defender* un sistema (o una red), resulta fundamental comprender cuáles son los vectores de ataque más comunes y –nuevamente– la relación entre los componentes involucrados para poder remediar o, mejor aún, prevenir dichos ataques.

Y claro está, puede verse al mundo en general, fuera del entorno del cómputo, como una serie de modelos interactuantes. Muchos de los métodos y algoritmos que se abordan en esta obra pueden emplearse fuera del entorno del cómputo; una vez comprendidos los problemas de concurrencia, de competencia por recursos, o de protección y separación que han sido resueltos en el campo de los sistemas operativos, estas soluciones pueden ser extrapoladas a otros campos.

El camino por delante es largo, y puede resultar interesante y divertido.

Capítulo 2

Funciones y objetivos del sistema operativo

El sistema operativo es el único programa que interactúa directamente con el hardware de la computadora. Sus funciones primarias son:

Abstracción Los programas no deben tener que preocuparse de los detalles de acceso a hardware, o de la configuración particular de una computadora. El sistema operativo se encarga de proporcionar una serie de abstracciones para que los programadores puedan enfocarse en resolver las necesidades particulares de sus usuarios. Un ejemplo de tales abstracciones es que la información está organizada en *archivos* y *directorios* (en uno o muchos *dispositivos de almacenamiento*).

Administración de recursos Una sistema de cómputo puede tener a su disposición una gran cantidad de *recursos* (memoria, espacio de almacenamiento, tiempo de procesamiento, etc.), y los diferentes *procesos* que se ejecuten en él *compiten* por ellos. Al gestionar toda la asignación de recursos, el sistema operativo puede implementar políticas que los asignen de forma efectiva y acorde a las necesidades establecidas para dicho sistema.

Aislamiento En un sistema multiusuario y multitarea cada proceso y cada usuario no tendrá que preocuparse por otros que estén usando el mismo sistema —Idealmente, su *experiencia* será la misma que si el sistema estuviera exclusivamente dedicado a su atención (aunque fuera un sistema menos poderoso).

Para implementar correctamente las funciones de aislamiento hace falta que el sistema operativo utilice hardware específico para dicha protección.

Capítulo 3

Evolución de los sistemas operativos

No se puede comenzar a abordar el tema de los sistemas operativos sin revisar brevemente su desarrollo histórico. Esto no sólo permitirá comprender por qué fueron apareciendo determinadas características y patrones de diseño que se siguen empleando décadas más tarde, sino (como resulta particularmente bien ejemplificado en el discurso de recepción del premio Turing de Fernando Corbató, *Acerca de la construcción de sistemas que fallarán*, (Corbató 2007)), adecuar un sistema a un entorno cambiante, por mejor diseñado que éste estuviera, lleva casi inevitablemente a abrir espacios de comportamiento no previsto —el espacio más propicio para que florezcan los fallos. Conocer los factores que motivaron a los distintos desarrollos puede ayudar a prever y prevenir problemas.

3.1. Proceso por lotes (*batch processing*)

Los antecedentes a lo que hoy se conoce como sistema operativo pueden encontrarse en la automatización inicial del procesamiento de diferentes programas, surgida en los primeros centros de cómputo: cuando en los años cincuenta aparecieron los dispositivos perforadores/lectores de tarjetas de papel, el tiempo que una computadora estaba improductiva esperando a que estuviera lista una *tarea* (como se designaba a una ejecución de cada determinado programa) para poder ejecutarla disminuyó fuertemente ya que los programadores entregaban su lote de tarjetas perforadas (en inglés, *batches*) a los operadores, quienes las alimentaban a los dispositivos lectores, que lo cargaban en memoria en un tiempo razonable, iniciaban y monitoreaban la ejecución, y producían los resultados.

En esta primer época en que las computadoras se especializaban en tareas de cálculo intensivo y los dispositivos que interactuaban con medios externos eran prácticamente desconocidos, el papel del sistema *monitor* o *de control* era básicamente asistir al operador en la carga de los programas y las bibliotecas requeridas, la notificación de resultados y la contabilidad de recursos empleados para su cobro.

Los sistemas monitores se fueron sofisticando al implementar protecciones que evitaran la corrupción de *otros trabajos* (por ejemplo, lanzar erróneamente la instrucción *leer siguiente tarjeta* causaría que el siguiente trabajo encolado perdiera sus primeros caracteres, corrompiéndolo e impidiendo su ejecución), o que entraran en un ciclo infinito, estableciendo *alarmas* (*timers*) que interrumpirían la ejecución de un proceso si éste duraba más allá del tiempo estipulado. Estos monitores implicaban la modificación del hardware para considerar dichas características de seguridad —y ahí se puede hablar ya de la característica básica de gestión de recursos que identifica a los sistemas operativos.

Cabe añadir que el tiempo de carga y puesta a punto de una tarea seguía representando una parte importante del tiempo que la computadora dedicaba al procesamiento: un lector de cintas rápido procesaba del orden de cientos de caracteres por minuto, y a pesar de la lentitud relativa de las computadoras de los años cincuenta ante los estándares de hoy (se medirían por miles de instrucciones por segundo, KHz, en vez de miles de millones como se hace hoy, GHz), esperar cinco o diez minutos con el sistema completamente detenido por la carga de un programa moderadadamente extenso resulta a todas luces un desperdicio.

3.2. Sistemas en lotes con dispositivos de carga (*spool*)

Una mejora natural a este último punto fue la invención del *spool*: un mecanismo de entrada/salida que permitía que una computadora de propósito específico, mucho más económica y limitada, leyera las tarjetas y las fuera convirtiendo a cinta magnética, un medio mucho más rápido, teniéndola lista para que la computadora central la cargara cuando terminara con el trabajo anterior. Del mismo modo, la computadora central guardarba sus resultados en cinta para que equipos especializados la leyeran e imprimieran para el usuario solicitante.

La palabra *spool* (bobina) se tomó como *acrónimo inverso* hacia *Simultaneous Peripherial Operations On-Line*, operación simultánea de periféricos en línea.

3.3. Sistemas multiprogramados

A lo largo de su ejecución, un programa normalmente pasa por etapas con muy distintas características: durante un ciclo fuertemente dedicado al cálculo numérico, el sistema opera *limitado por el* CPU (CPU-*bound*), mientras que al leer o escribir resultados a medios externos (incluso mediante *spools*) el límite es impuesto por los dispositivos, esto es, opera *limitado por entrada-salida* (*I-O bound*). La programación multitareas o los sistemas multiprogramados buscaban maximizar el tiempo de uso efectivo del procesador ejecutando varios procesos al mismo tiempo.

El hardware requerido cambió fuertemente. Si bien se esperaba que cada usuario fuera responsable con el uso de recursos, resultó necesario que apareciera la infraestructura de protección de recursos: un proceso no debe sobreescribir el espacio de memoria de otro (ni el código, ni los datos), mucho menos el espacio del monitor. Esta protección se encuentra en la *Unidad de Manejo de Memoria* (MMU), presente en todas las computadoras de uso genérico desde los años noventa.

Ciertos dispositivos requieren bloqueo para ofrecer acceso exclusivo/único: cintas e impresoras, por ejemplo, son de acceso estrictamente secuencial, y si dos usuarios intentaran usarlas al mismo tiempo, el resultado para ambos se corrompería. Para estos dispositivos, el sistema debe implementar otros *spools* y mecanismos de bloqueo.

3.4. Sistemas de tiempo compartido

El modo de interactuar con las computadoras se modificó drásticamente durante los años sesenta, al extenderse la multitarea para convertirse en sistemas *interactivos* y *multiusuarios*, en buena medida diferenciados de los anteriores por la aparición de las *terminales* (primero teletipos seriales, posteriormente equipos con una pantalla completa como se conocen hasta hoy).

En primer término, la tarea de programación y depuración del código se simplificó fuertemente al poder hacer el programador directamente cambios y someter el programa a la ejecución inmediata. En segundo término, la computadora *nunca más estaría simplemente esperando a que esté listo un progama*: mientras un programador editaba o compilaba su programa, la computadora seguía calculando lo que otros procesos requirieran.

Un cambio fundamental entre el modelo de *multiprogramación* y de *tiempo compartido* es el tipo de control sobre la multitarea (se verá en detalle en el capítulo III).

Multitarea *cooperativa* o *no apropiativa* (*Cooperative multitasking*). La implementaron los sistemas multiprogramados: cada proceso tenía control del CPU hasta que éste hacía una llamada al sistema (o indicara su *disposición a cooperar* por medio de la llamada `yield`: *ceder el paso*).

Un cálculo largo no era interrumpido por el sistema operativo, en consecuencia un error de programador podía congelar la computadora completa.

Multitarea *preventiva* o *apropiativa* (*Preemptive multitasking*). En los sistemas de tiempo compartido, el reloj del sistema interrumpe periódicamente a los diversos procesos, transfiriendo *forzosamente* el control nuevamente al sistema operativo. Éste puede entonces elegir otro proceso para continuar la ejecución.

Además, fueron naciendo de forma natural y paulatina las abstracciones que se conocen hoy en día, como los conceptos de *archivos* y *directorios*, y el código necesario para emplearlos iba siendo enviado a las *bibliotecas de sistema* y, cada vez más (por su centralidad) hacia el núcleo mismo del, ahora sí, sistema operativo.

Un cambio importante entre los sistemas multiprogramados y de tiempo compartido es que la velocidad del cambio entre una tarea y otra es mucho más rápido: si bien en un sistema multiprogramado un *cambio de contexto* podía producirse sólo cuando la tarea cambiaba de un modo de ejecución a otro, en un sistema interactivo, para dar la *ilusión* de uso exclusivo de la computadora, el hardware emitía periódicamente al sistema operativo *interrupciones* (señales) que le indicaban que cambie el *proceso* activo (como ahora se le denomina a una instancia de un programa en ejecución).

Diferentes tipos de proceso pueden tener distinto nivel de importancia —ya sea porque son más relevantes para el funcionamiento de la computadora misma (procesos de sistema), porque tienen mayor carga de interactividad (por la experiencia del usuario) o por diversas categorías de usuarios (sistemas con contabilidad por tipo de atención). Esto requiere la implementación de diversas *prioridades* para cada uno de éstos.

Capítulo 4

Y del lado de las computadoras personales

Si bien la discusión hasta este momento asume una computadora central con operadores dedicados y múltiples usuarios, en la década de los setenta comenzaron a aparecer las *computadoras personales*, sistemas en un inicio verdaderamente reducidos en prestaciones y a un nivel de precios que los ponían al alcance, primero, de los aficionados entusiastas y, posteriormente, de cualquiera.

4.1. Primeros sistemas para entusiastas

Las primeras computadoras personales eran distribuidas sin sistemas operativos o lenguajes de programación; la interfaz primaria para programarlas era mediante llaves (*switches*), y para recibir sus resultados, se utilizaban bancos de LEDs. Claro está, esto requería conocimientos especializados, y las computadoras personales eran aún vistas sólo como juguetes caros.

4.2. La revolución de los 8 bits

La verdadera revolución apareció cuando, poco tiempo más tarde, comenzaron a venderse computadoras personales con salida de video (típicamente por medio de una televisión) y entrada por un teclado. Estas computadoras popularizaron el lenguaje BASIC, diseñado para usuarios novatos en los sesenta, y para permitir



Figura 4.1: La *microcomputadora Altair 8800*, primera computadora personal con distribución masiva, a la venta a partir de 1975 (imagen de la Wikipedia: *Altair 8800*).

a los usuarios gestionar sus recursos (unidades de cinta, pantalla posicionable, unidades de disco, impresoras, modem, etc.) llevaban un software mínimo de sistema —nuevamente, un proto-sistema operativo.



Figura 4.2: La *Commodore Pet 2001*, en el mercado desde 1977, una de las primeras con intérprete de BASIC (imagen de la Wikipedia: *Commodore PET*).

4.3. La computadora para fines "serios": la familia PC

Al aparecer las computadoras personales "serias", orientadas a la oficina más que al *hobby*, a principios de los ochenta (particularmente representadas por la IBM PC, 1981), sus sistemas operativos se comenzaron a diferenciar de los equipos previos al separar el *entorno de desarrollo* en algún lenguaje de programación del *entorno de ejecución*. El papel principal del sistema operativo ante el usuario era administrar los archivos de las diversas aplicaciones mediante una sencilla interfaz de línea de comando, y lanzar las aplicaciones que el usuario seleccionaba.

La PC de IBM fue la primer arquitectura de computadoras personales en desarrollar una amplia familia de *clones*, computadoras compatibles diseñadas para trabajar con el mismo sistema operativo, y que eventualmente capturaron casi 100% del mercado. Prácticamente todas las computadoras de escritorio y portátiles en el mercado hoy derivan de la arquitectura de la IBM PC.

Ante las aplicaciones, el sistema operativo (PC-DOS, en las versiones distribuidas directamente por IBM,



Figura 4.3: La computadora IBM PC modelo 5150 (1981), iniciadora de la arquitectura predominantemente en uso hasta el día de hoy (imagen de la Wikipedia: *IBM Personal Computer*).

o el que se popularizó más, MS-DOS, en los *clones*) ofrecía la ya conocida serie de interfaces y abstracciones para administrar los archivos y la entrada/salida a través de sus puertos. Cabe destacar que, particularmente en sus primeros años, muchos programas se ejecutaban directamente sobre el hardware, arrancando desde el BIOS y sin emplear el sistema operativo.

4.4. El impacto del entorno gráfico (WIMP)

Hacia mediados de los ochenta comenzaron a aparecer computadoras con interfaces usuario gráficas (*Graphical User Interfaces*, *textscguis*) basadas en el paradigma WIMP (*Windows, Icons, Menus, Pointer*; Ventanas, Iconos, Menúes, Apuntador), que permitían la interacción con varios programas al mismo tiempo. Esto *no necesariamente* significa que sean sistemas multitarea: por ejemplo, la primer interfaz de MacOS permitía ver varias ventanas abiertas simultáneamente, pero sólo el proceso activo se ejecutaba.

Esto comenzó, sin embargo, a plantear inevitablemente las necesidades de concurrencia a los programadores. Los programas ya no tenían acceso directo a la pantalla para manipular a su antojo, sino que a una abstracción (la *ventana*) que podía variar sus medidas, y que requería que toda la salida fuera estrictamente mediante las llamadas a bibliotecas de primitivas gráficas que comenzaron a verse como parte integral del sistema operativo.

Además, los problemas de protección y separación entre procesos concurrentes comenzaron a hacerse evidentes: los programadores tenían ahora que programar con la conciencia de que compartirían recursos, con el limitante (que no tenían en las máquinas *profesionales*) de no contar con hardware especializado para esta protección. Los procesadores en uso comercial en los ochenta no manejaban *anillos* o *niveles de ejecución* ni *unidad de administración de memoria* (MMU), por lo que un programa fallado o dañino podía corromper la operación completa del equipo. Y si bien los entornos que más éxito tuvieron (Apple MacOS y Microsoft Windows) no implementaban multitarea real, sí hubo desde el principio sistemas como la Commodore Amiga o la Atari ST que hacían un multitasking *apropiativo* verdadero.



Figura 4.4: Apple Macintosh (1984), popularizó la interfaz usuario gráfica (GUI) (imagen de la Wikipedia: *Macintosh*).

Naturalmente, ante el uso común de un entorno de ventanas, los programas que se ejecutaban sin requerir de la carga del sistema operativo cayeron lentamente en el olvido.

4.5. Convergencia de los dos grandes mercados

Conforme fueron apareciendo los CPU con características suficientes en el mercado para ofrecer la protección y aislamiento necesario (particularmente, Intel 80386 y Motorola 68030), la brecha de funcionalidad entre las computadoras personales y las *estaciones de trabajo* y *mainframes* se fue cerrando.

Hacia principios de los 1990, la mayor parte de las computadoras de arquitecturas *alternativas* fueron cediendo a las presiones del mercado, y hacia mediados de la década sólo quedaban dos arquitecturas principales: la derivada de IBM y la derivada de la Apple Macintosh.

Los sistemas operativos primarios para ambas plataformas fueron respondiendo a las nuevas características del hardware: en las IBM, la presencia de Microsoft Windows (originalmente un *entorno operativo* desde su primera edición en 1985, evolucionando hacia un sistema operativo completo ejecutando sobre una base de MS-DOS en 1995) se fue haciendo prevalente hasta ser la norma. Windows pasó de ser un sistema meramente de aplicaciones propias y que operaba únicamente por reemplazo de aplicación activa a ser un sistema de multitarea cooperativa y, finalmente un sistema que requería protección en hardware (80386) e implementaba multitarea apropiativa.

A partir del 2003, el núcleo de Windows en más amplio uso fue reemplazado por un desarrollo hecho de inicio como un sistema operativo completo y ya no como un programa bajo MS-DOS: el núcleo de nueva tecnología (Windows NT), que, sin romper compatibilidad con los APIs históricos de Windows, ofreció mucho mayor estabilidad.



Figura 4.5: Commodore Amiga 500 (1987), la computadora más popular de la familia *Amiga*, con amplias capacidades multimedia y multitarea apropiativa; una verdadera maravilla para su momento (imagen de la Wikipedia: *Amiga*).

Por el lado de Apple, la evolución fue muy en paralelo: ante un sistema ya agotado y obsoleto, el MacOS 9, en 2001 anunció una nueva versión de su sistema operativo que fue en realidad un relanzamiento completo: MacOS X es un sistema basado en un núcleo Unix BSD, sobre el *microkernel* Mach.

Y otro importante jugador que entró en escena durante los años noventa fue el software libre, por medio de varias implementaciones distintas de sistemas tipo Unix, principalmente, Linux y los *BSD (FreeBSD, NetBSD, OpenBSD). Estos sistemas implementaron, colaborativamente y bajo un esquema de desarrollo geográficamente distribuido, software compatible tanto con las PC como con el que se ejecutaba en las estaciones de trabajo a gran escala, con alta confiabilidad, y cerrando por fin la divergencia del árbol del desarrollo de la computación en *fierros grandes* y *fierros chicos*.

Al día de hoy, la arquitectura derivada de Intel (y la PC) es el claro ganador de este proceso de 35 años, habiendo conquistado casi la totalidad de los casos de uso, incluso las máquinas Apple. Hoy en día, la arquitectura Intel ejecuta desde subportátiles hasta supercomputadoras y centros de datos; el sistema operativo específico varía según el uso, yendo mayoritariamente hacia Windows, con los diferentes Unixes concentrados en los equipos servidores.

En el frente de los dispositivos *embebidos* (las computadoras más pequeñas, desde microcontroladores hasta teléfonos y tabletas), la norma es la arquitectura ARM, también bajo versiones específicas de sistemas operativos Unix y Windows (en ese orden).

Capítulo 5

Dispositivos móviles

En los últimos años, buena parte del desarrollo en el mundo del cómputo se ha volcado hacia el modelo de cómputo representado, genéricamente, por los *dispositivos móviles*. Dado el interés que estas plataformas han despertado, se torna necesario abordar el tema, aunque sea más para anotar similitudes que diferencias con el resto de los equipos de cómputo. Para hacer esto, sin embargo, es necesario primero abordar la definición: ¿en qué consiste un *dispositivo móvil*, cuáles son los límites de su definición, qué fronteras se le pueden definir?

Es difícil encontrar límites claros y duros para lo que este concepto abarca; en el transcurso de esta sección se abordan las características de las computadoras diseñadas no sólo en el nivel del hardware, sino de interfaz usuario, para que su propietario las cargue consigo y las convierta en un asistente para sus actividades cotidianas, para la organización de su vida diaria. Partiendo de esta definición se tiene que un *teléfono inteligente* será tratado como dispositivo móvil, pero una computadora portátil no, puesto que su interfaz es la misma de una computadora estándar.

Claro, esta definición –indudablemente rápida e imperfecta– deja una gran área gris, y permite cierta ambigüedad. Por ejemplo, las más recientes versiones de algunos entornos de usuario (notablemente, la interfaz primaria de Windows 8, o los entornos GNOME y Unity de Linux) buscan *unificar la experiencia*, incorporando conceptos del *multitouch* a los escritorios y acercando los casos de uso. Tómense, pues, estos lineamientos como meramente indicativos.

5.1. Reseña histórica

Tener una plataforma de cómputo móvil ha sido uno de los anhelos más reiterados del cómputo; ya en 1975, antes de la aparición de todos los sistemas reseñados en la sección 4 (a excepción de la Altair 8800) IBM lanzó al mercado su primer computadora portátil: La IBM 5100, de 25 Kg de peso y con una pantalla de 5 pulgadas (equivalente a un teléfono celular grande del día de hoy). Esta computadora tuvo un éxito muy limitado en buena medida por su precio: 9 000 dólares en su configuración más básica la dejaban claramente fuera del alcance del mercado de los entusiastas de la época, y la incomodidad de su pequeña pantalla llevó al entorno corporativo a preferir seguir usando las minicomputadoras con terminales estándar para la época.

Este mercado también vio una importante convergencia, en este caso *desde abajo*: la miniautrización vivida en la década de los setenta fue, a fin de cuentas, iniciada por el CPU Intel 4004, diseñado expresamente para las calculadoras Busicom. Durante esa época nacieron las calculadoras portátiles. Éstas comenzaron implementando únicamente las operaciones aritméticas básicas, pero con el paso del tiempo aparecieron las *calculadoras científicas*, incluyendo operaciones trigonométricas. En 1974, Hewlett-Packard lanzó al mercado la HP-65 la primer calculadora de bolsillo plenamente programable.

Para 1984, ya ante la franca popularización de las aplicaciones ofimáticas, la empresa británica *Psion* lanzó la *Psion Organiser*, que se anunciaba como la primer computadora de bolsillo práctica del mundo: era vendida con reloj, calculadora, una base de datos sencilla, y cartuchos de expansión con aplicaciones ejemplo (ciencia, matemáticas y finanzas), además de un entorno de programación para que el usuario desarrollara sus propias aplicaciones.

El hardware del *Organiser* original era, claro está, muy limitado. Con sólo 4 KB de ROM y 2 KB de memoria no incluía un sistema operativo, y el lenguaje de programación disponible al usuario era meramente un en-



Figura 5.1: Psion Organiser, anunciada como *la primer computadora de bolsillo práctica del mundo* en 1984. En la imagen, un dispositivo de su segunda generación (imagen de la Wikipedia: *Psion Organiser*).

samblador. No tener un sistema operativo significa que, en vez de hacer las *llamadas al sistema* necesarias para realizar transferencias (como se verá en la secc. 15 y en el cap. VII), el programador tenía que avanzar y transferir *byte por byte*. Dos años más tarde, la segunda generación del *Organiser* salió al mercado con un sistema operativo monotarea y mucho mayor espacio de almacenamiento. Varias generaciones más tarde, este sistema operativo es el que hacia 1998 se convirtió en *Symbian*, que fuera el dominante del mercado de celulares durante la mayor parte de la década del 2000.



Figura 5.2: Sharp ZQ-770, diseño bajo uno de los formatos de PDA (Asistente Personal Digital) más popularizados de la década de los noventa (imagen de la Wikipedia: *Sharp Wizard*).

Siguiendo los pasos del *Organiser*, muchas otras empresas fueron creando pequeños equipos con aproximadamente la misma funcionalidad básica (lista de contactos, notas y agenda) e interfaz usuario, definiendo el término de *Asistente Digital Personal* (*Personal Digital Assistant*, PDA). Hubo diferentes hitos durante la década de los noventa, aunque destaca particularmente la plataforma *Palm*. Esta fue la primera plataforma con éxito al incorporar una interfaz usuario táctil con escritura basada en reconocimiento de la letra (que era *trazada* por medio de una pluma especial, o *stylus*, en la pantalla).

El siguiente paso natural fue unir la funcionalidad del cada vez más popular teléfono celular con la del PDA. Ya desde 1996 se comercializaron equipos ofreciendo la funcionalidad integrada, y el término *smartphone* (*teléfono inteligente*) se empleó por primera vez en 1998. Como se verá en la sección 5.2, el reto de mantener la

señalización estable significó que muchos de estos teléfonos resultaban en una suerte de *Frankenstein*, con dos *personalidades* claramente diferenciadas.



Figura 5.3: El *iPhone*, de Apple, introdujo la primera interfaz usuario *multitouch* y detonó la popularidad de los *teléfonos inteligentes* —y con ello, del cómputo móvil (imagen de la Wikipedia: *iPhone 2*).

En el año 2007, Apple presentó su hoy icónico *iPhone*. Desde un punto de vista técnico, la principal innovación de este equipo fue una nueva interfaz gráfica denominada *multitouch* (*multitoque*), que permite al usuario interactuar directamente con sus dedos (por medio de toques combinados o *gestos* y ya no requiriendo de un *stylus*) e incluso de la inclinación del dispositivo. Y si bien el teléfono mismo no representó un salto en las capacidades del hardware, Apple logró diseñar una interfaz innovadora –como ya lo había hecho en 1984 con la Macintosh– que se convirtió rápidamente en estándar para todo un mercado.

Hasta este punto, prácticamente la totalidad de dispositivos en el segmento reconocido como móvil eran reconocibles por su tamaño: casi todos los dispositivos mencionados en esta sección están hechos para caber en el bolsillo de una camisa. Sin embargo, a este segmento deben agregarse las *tabletas*. La historia de su desarrollo y adopción se parecen a la aquí presentada respecto a la interfaz de los teléfonos inteligentes (e incluso, llevada más al extremo): Es posible encontrar antecedentes desde 1915,[1] numerosas descripciones literarias en la ciencia ficción a lo largo del siglo XX, y varias implementaciones funcionales desde inicios de la década de los noventa. Sin embargo, las tabletas parecieron por largos años estar destinadas a nunca conquistar al mercado, hasta el año 2010, en que Apple lanzó un equipo con la misma interfaz de su *iPhone* pero del tamaño de una computadora portátil estándar.

Todos los sistemas disponibles hoy en día, claro está, tienen muchísima mayor complejidad que la del *Psion Organizer*, y hay varias familias de sistemas operativos de uso frecuente; se describirán a continuación, y muy a grandes rasgos, sólo algunos de los sistemas en uso. En la presente sección se enumeran únicamente con su información general, y en la siguiente se mencionan algunas de sus características técnicas.

iOS El sistema operativo de Apple, y diseñado exclusivamente para el hardware producido por dicha compañía. Fue el primero en implementar la interfaz usuario *multitouch* y, en buena medida, se puede ver como el responsable de la explosión y universalización en el uso de dispositivos móviles. Al igual que el sistema operativo que emplean para sus equipos de escritorio, MacOS X, iOS está basado en el núcleo *Darwin*, derivado de FreeBSD, un sistema libre tipo Unix.

Android Diseñado por la compañía *Google*, basa la mayor parte de su operación en software libre (un núcleo Linux, máquina virtual Java, y muchas de las bibliotecas de sistema comunes en sistemas Linux),

[1] El registro de patente 1 117 184 de los Estados Unidos (Goldberg 1914) se refiere a una máquina para reconocer los caracteres escritos en una hoja.

agregando una capa de servicios propietarios. La estrategia de Google ha sido inversa a la de Apple: en vez de fabricar sus propios dispositivos, otorga licencias para el uso de este sistema operativo a prácticamente todos los fabricantes de hardware, con lo que la amplia mayoría de los modelos de teléfonos inteligentes y tabletas corren sobre Android.

Windows Phone Microsoft ofrece una versión de su sistema operativo, compatible en API con el Windows de escritorio, pero compilado para procesador ARM. Este sistema operativo no ha logrado conquistar gran popularidad, en claro contraste con su dominación en el cómputo *tradicional* de escritorio; el principal fabricante que vende equipos con *Windows Phone* es Nokia (que, después de haber sido la compañía líder en telefonía, fue adquirida por Microsoft mismo).

Symbian Si bien este sistema operativo ya está declarado como oficialmente muerto, su efecto en el desarrollo temprano del segmento fue fundamental, y no puede ser ignorado. Symbian fue la plataforma principal para Nokia en su época de gloria, así como para muchos otros fabricantes. Casi todas las empresas que antiguamente operaban con Symbian han mudado su oferta a sistemas Android.

Firefox OS La fundación Mozilla, responsable del navegador Firefox (y heredera del histórico Netscape) está intentando entrar al mercado móbil con este sistema, basado (al igual que Android) en el núcleo de Linux, pero orientado a ofrecer una interfaz de programación siguiendo completamente los estándares y lenguajes para uso en la Web. Esta plataforma hace una apuesta mucho más agresiva que las demás a un esquema de conexión permanente a la red de datos.

5.2. Características diferenciadoras

Resultará claro, a partir de los sistemas recién presentados, así como la gran mayoría de los sistemas operativos empleados para dispositivos móviles, que la diferenciación entre el segmento móvil y el cómputo tradicional *no está* en el sistema operativo mismo, sino en capas superiores. Sin embargo, la diferencia va mucho más allá de un cambio en la interfaz usuario; las características de estos dispositivos indudablemente determinan cuestiones de fondo. A continuación, se exponen algunas de las características más notorias.

5.2.1. Almacenamiento en estado sólido

La primer característica notoria al manipular un teléfono o una tableta es que ya no se hace con la noción de fragilidad que siempre acompañó al cómputo: los discos duros son dispositivos de altísima precisión mecánica, y un pequeño golpe puede significar su avería absoluta y definitiva. Los dispositivos móviles operan con almacenamiento en *estado sólido*, esto es, en componentes electrónicos sin partes móviles. La evolución y las características del almacenamiento en estado sólido serán cubiertas en la sección K.2.

Al estar principalmente orientados a este medio de almacenamiento, en líneas generales, los sistemas operativos móviles no emplean memoria virtual, tema que será cubierto en la sección 34. No pueden, por tanto, mantener en ejecución programas que excedan del espacio *real* de memoria con que cuente el sistema — y esto conlleva importantes consideraciones de diseño.

5.2.2. Multitarea, pero *monocontexto*

La forma de uso de las computadoras dio un salto cualitativo, tanto en el mundo corporativo hacia la década de los sesenta como en el personal hacia inicios de los noventa, con la introducción de sistemas con capacidades multitarea (véase la sección 3.3). Los usuarios se han acostumbrado a que sus equipos hagan muchas cosas (aparentemente) al mismo tiempo, y es ya una expectativa común el poder tener abierta una cantidad arbitraria, a veces incluso excesiva, de programas en ejecución, al grado de que prácticamente los

usuarios promedio del cómputo reconocen perfectamente la *hiperpaginación* (que será descrita en la sección 34.5) por sus síntomas.

La popularización del cómputo móvil llevó, sin embargo, a una fuerte *reducción* en las expectativas de multitarea. Esto principalmente por dos razones; la primera es que, al carecer los dispositivos móviles de memoria virtual, la memoria disponible se vuelve nuevamente un bien escaso, y el sistema operativo se ve obligado a limitar al número de *procesos interactivos* en ejecución.[2]

Esta distinción también puede explicarse por el modelo de uso de estos dispositivos: comparadas con las pantallas de equipos de escritorio, donde las pantallas más frecuentemente utilizadas son de 17 pulgadas,[3] los teléfonos van en general de las 3.5 a las 5 pulgadas. Una interfaz usuario diseñada para un tipo de pantalla no puede resultar satisfactoria en el otro.

Las interfaces usuario empleadas por los sistemas móviles abandonan el modelo de interacción WIMP presentado en la sección 4.4, así como la *metáfora del escritorio*, para volver a la de un sólo programa visible en todo momento.

Al buscar satisfacer las necesidades de un mercado mucho más amplio y mucho menos versado en los detalles del cómputo, todas estas interfaces conllevan importante simplificaciones. Una de las más notorias es que los usuarios ya no solicitan la finalización de un programa: los programas van siendo lanzados (y utilizados uno por uno), y *si caben en memoria* son mantenidos abiertos para evitar las demoras de volver a inicializar. El sistema define políticas por medio de las cuales estos programas serán finalizados y *evacuados* de la memoria al llegar a determinados umbrales.

5.2.3. Consumo eléctrico

Una de las áreas en que más visible ha sido el desarrollo cualitativo durante los últimos años es la optimización del consumo eléctrico de los equipos de cómputo. Y si bien no puede negarse la importancia del ahorro eléctrico en las oficinas y centros de datos, donde el trabajo diario cada vez depende más del cómputo, tampoco puede ignorarse la importancia de popularización de las computadoras portátiles. Y agregando algunos patrones particulares a estos dispositivos, la popularización del cómputo móvil ha llevado a una verdadera revolución en este aspecto.

El ahorro del consumo eléctrico tiene dos principales vertientes: por un lado, el desarrollo de hardware más eficiente energéticamente, con independencia del modo en que opere y, por el otro, la creación de mecanismos por medio de los cuales un equipo de cómputo pueda detectar cambios en el patrón de actividad (o el operador pueda indicarle un cambio en la respuesta esperada), y éste reaccione reduciendo su demanda (lo cual típicamente se obtiene reduciendo la velocidad de ciertos componentes del equipo).

El primer esquema de ahorro de energía con amplio soporte, tanto por parte de hardware de diversos fabricantes, como de prácticamente todos los sistemas operativos ampliamente utilizado, fue APM (*Advanced Power Management*, Gestión Avanzada de la Energía). Pasado cierto tiempo, fue reemplazado por ACPI (*Advanced Configuration and Power Interface*, Interfaz Avanzada de Configuración y Energía); la principal diferencia entre ambas es que, mientras que bajo APM los diferentes niveles de energía se implementaban en el *firmware* de la computadora o cada uno de sus dispositivos, en ACPI la responsabilidad recae en el sistema operativo; esto brinda mucho mayor flexibilidad a la implementación, a cambio de una mayor complejidad para los desarrolladores.

[2] Formalmente, no es el sistema operativo mismo, sino que el *software de sistema*, que monitorea el uso y rendimiento, y toma decisiones a más alto nivel. En todas las plataformas mencionadas, hay una fuerte distinción entre los programas que operan como *servicios* y se mantienen activos en el fondo y aquellos que operan en primer plano, con interfaz gráfica e interacción directa con el usuario.

[3] La medida más habitual para las pantallas indica las pulgadas que miden *en diagonal*. Tras muchos años en que la relación de dimensión vertical contra horizontal o *aspecto* de las pantallas fuera de 3:4, este formato se ha ido reemplazando por el de 9:16, por lo que la medida en pulgadas por sí sola ahora lleva una carga de ambigüedad.

Pero la verdadera diferencia en el tema que esta sección aborda es la frecuencia de los cambios de estado: un servidor o computadora de escritorio tiene sólo un *evento* constante (el ajuste de frecuencia del procesador dependiendo de la carga, potencialmente hasta decenas de veces por segundo); una computadora portátil debe adoptar diferentes *perfiles* dependiendo de si está conectada a la red eléctrica u operando por batería, o si tiene la *tapa* (pantalla) abierta o cerrada. Los usuarios de computadoras portátiles típicamente buscan trabajar conectados a la red eléctrica tanto como sea posible, dado que es *vox populi* que esto mejorará la vida útil de su batería. Y si bien estos cambios de entorno se presentan (y guardan una innegable complejidad), su ocurrencia es muy baja.

En el cómputo móvil, los eventos son muchos y muy distintos. En primer lugar, los dispositivos móviles operan bajo una filosofía de *siempre encendido*: A pesar de que el usuario no esté atento a su dispositivo, éste tiene que estar encendido y al pendiente del entorno —algunos ejemplos casi obvios:

- En caso de entrar una llamada telefónica, tiene que responder inmediatamente alertando al usuario. La interfaz usuario del teléfono puede parecer apagada, pero su lógica (y en particular su señalización a las distintas redes a las que está conectado) se mantiene activa.
- El equipo tiene que estar siempre alerta a las condiciones cambiantes de red (tanto telefónica como de datos), midiendo la señal de las antenas celulares más cercanas; mantenerse asociado a una antena remota requiere más energía que a una cercana.
- Dado que estos equipos están diseñados para *moverse* junto con el usuario, convirtiéndose en un asistente o una extensión para sus actividades diarias, optimizan su funcionamiento para operar como norma desde su batería, no conectados a la red eléctrica. Valga en este caso la comparación: un teléfono que brinde menos de un día de *operación autónoma* sería sin duda evaluado como extremadamente incómodo e impráctico, en tanto una computadora portátil se considera muy eficiente si permite la operación autónoma por seis horas.

El ahorro de energía que permite estos patrones de uso no sólo se debe al hardware cada vez más eficiente que emplean los dispositivos móviles, sino que a una programación de las aplicaciones en que los desarrolladores explícitamente buscan patrones eficientes, fácil *suspensión*, y minimizando la necesidad de *despertar* al hardware.

5.2.4. Entorno cambiante

Los centros de datos, las computadoras de escritorio, e incluso las portátiles tienen una forma de operación bastante estable: durante el transcurso de una sesión de trabajo (e incluso durante la vida entera del equipo, en el caso de los servidores) su visión del mundo no está sujeta a mayores cambios. Un usuario puede mantener por largos periodos su *configuración de consumo energético*. Las interfaces y direcciones de red son típicamente estables, y si hubiera un problema de señal, la reparación o reubicación se haría en la infraestructura de red, no en el equipo cliente. El formato de la pantalla, claro está, es también estable.

Uno de los cambios más difíciles de implementar en el software del sistema fue precisamente el de brindar la plasticidad necesaria en estos diferentes aspectos: el dispositivo móvil debe ser más enérgico en sus cambios de perfil de energía, respondiendo a un entorno cambiante. Puede aumentar o disminuir la luminosidad de la pantalla dependiendo de la luminosidad circundante, o desactivar determinada funcionalidad si está ya en niveles críticos de carga. Con respecto a la red, debe poder aprovechar las conexiones fugaces mientras el usuario se desplaza, iniciando eventos como el de *sincronización*. Y encargándose de detener (tan limpiamente como sea posible) los procesos que van dejando de responder. Por último, claro, la interfaz usuario: los dispositivos móviles no tienen una orientación # única natural, como sí la tienen las computadoras. Las interfaces usuario deben pensarse para que se puedan *reconfigurar* ágilmente ante la rotación de la pantalla.

5.2.5. El *jardín amurallado*

Una consecuencia indirecta (y no técnica) del nacimiento de las plataformas móviles es la popularización de un modelo de distribución de software conocido como *jardín amurallado* o, lo que es lo mismo, una *plataforma cerrada*.

Partiendo de que los teléfonos inteligentes, en un primer momento, y las tabletas y dispositivos similares posteriormente, buscan satisfacer un mercado mucho mayor al de los entusiastas del cómputo,[4] Apple anunció en julio del 2008 (un año después del lanzamiento del iPhone) su tienda de aplicaciones o *app store*. La peculiaridad de ésta con relación al modelo de cómputo que ha imperado históricamente es que, si bien cualquier desarrollador puede crear una aplicación y enviarla, Apple se reserva el derecho de aprobarla, o eliminarla en cualquier momento. Esto es, este modelo le permite erigirse en juez, determinando qué puede o no ejecutar un usuario.

Este mismo modelo fue adoptado por Google para su sistema Android, en un principio bajo el nombre *Mercado Android*, y desde el 2012 como *Google Play*. Microsoft hizo lo propio con su *Windows Phone Store*.

Este modelo de autorización y distribución de software, sin embargo, rompe con lo que Jonathan Zittrain (2008) define como la *generatividad* de los equipos de cómputo y de la red en general. Para ampliar el debate en este entido, el libro de Zittrain se ha vuelto referencia obligada, y está disponible completo en línea.

Capítulo 6

Seguridad informática

No puede perderse de vista la importancia de la *seguridad informática*. Puede verse el efecto de este concepto en prácticamente todos los componentes que conforman a un sistema operativo. Para no ir más lejos, las funciones principales presentadas en la sección 2 cruzan necesariamente por criterios de seguridad. Algunas consideraciones podrían ser, por ejemplo:

Abstracción El sistema operativo debe asegurarse no sólo de proveer las abstracciones necesarias, sino también de que ninguno de sus usuarios pueda *evadir* dichas abstracciones. Por ejemplo, el que un usuario tenga derecho a modificar un archivo que esté alojado en determinada unidad de disco, no debe poder escribir *directamente* al disco; su acceso debe estar limitado a la interfaz que el sistema le ofrece.

Administración de recursos Si el sistema operativo definió determinada política de asignación de recursos, debe evitar que el usuario exceda las asignaciones aceptables, sea en el curso de su uso normal, o incluso ante patrones de uso *oportunista* — Esto es, conociendo los mecanismos y políticas, un usuario no debe poder lograr que el sistema le permite el uso por encima de lo definido.

Aislamiento Si el sistema operativo ofrece separación entre los datos, procesos y recursos de sus distintos usuarios, ninguno de ellos debe –accidental o intencionalmente– tener acceso a la información que otro

[4] Y también como respuesta a que algunos usuarios encontraron cómo romper la protección y poder desarrollar e instalar aplicaciones extraoficiales en esta plataforma, diseñada originalmente para sólo correr software de Apple.

haya marcado como privada. Además, retomando el inciso anterior, ninguno de los usuarios debe poder lograr que, por sus acciones, el sistema *penalice* a otros más allá de lo que la política de asignación de recursos estipule.

Claro está, estos tres incisos son presentados únicamente como ejemplo; a lo largo de la obra se presentarán varios casos relacionados con los distintos temas que se abordan.

Naturalmente, todo problema que se plantee relativo a la seguridad informática puede ser abordado (por lo menos) desde dos puntos de vista antagonistas: el de la *protección*, que busca definir y proteger los aspectos en que intervenga la seguridad para un problema dado, y el del *ataque*, que busca las debilidades o *vulnerabilidades* en determinada implementación de un esquema de seguridad que permitan, a quien las conozca, violar los límites que el administrador del sistema busca imponerle. Y claro, si bien el tipo de análisis para estos puntos de vista es muy distinto, comprender ambos resulta no únicamente legítimo sino necesario para una formación profesional completa.

Todas las áreas que aborda la presente obra tienen aspectos que pueden analizarse desde la seguridad informática, y más que dedicar un capítulo en particular a este tema, la apuesta es por abordar la seguridad de forma transversal.

6.1. Código malicioso

Los sistemas operativos, al igual que todo programa de cómputo, presentan imperfecciones, errores u omisiones, tanto en su diseño como en su implementación. El *código malicioso* (también conocido como *malware*) consiste en programas diseñados para *aprovechar* dichas *vulnerabilidades* para adquirir privilegios de ejecución o acceso a datos que de otro modo no habrían logrado.

Si la vulnerabilidad que aprovecha el código malicioso es resultado de un error en la implementación, el desarrollador del sistema operativo típicamente podrá corregirla y poner esta corrección (coloquialmente denominada *parche*) a disposición de los usuarios; en los sistemas operativos modernos, la instalación de estas correcciones se efectúa de forma automatizada. Por otro lado, si la vulnerabilidad es consecuencia de una debilidad en el *diseño*, su corrección puede ser mucho más compleja, incluso puede ser imposible de resolver, como el caso presentado en la sección 35.1.[1]

Cabe mencionar que una gran cantidad de código malicioso ataca a una capa particularmente débil de todo sistema de cómputo: al usuario. Un aspecto frecuente (y de muy difícil solución) de estos programas es que *engañan* al usuario presentándose como código legítimo, y si éste reacciona como el código malicioso busca, le permitirá la ejecución en el sistema con sus privilegios.

El código malicioso tiende a agruparse y clasificarse de acuerdo a su comportamiento, particularmente de cara al usuario: *virus*, *gusanos*, *caballos de troya*, *exploits*, y muchos más. Sin embargo, y dado que sus diferencias radican particularmente en sus múltiples comportamientos *ante el usuario* o *como programa en ejecución* (y se comportan en líneas generales del mismo modo ante el sistema operativo), se determinó que entrar en detalles al respecto resultaría fuera del ámbito de la presente obra.

[1] El caso de los desbordamientos de buffer no se debe directamente al diseño de uno de los sistemas operativos en particular. Su existencia, así como su presencia generalizada por más de 40 años después de haberse descrito, puede explicarse por la imposibilidad de resolverse este problema sin el consumo reiterado de recursos de cómputo con operaciones demasiado frecuentes. En este caso en particular, el desbordamiento puede evitarse únicamente usando lenguajes con gestión automática de memoria, mucho más lentos que los lenguajes de bajo nivel, o concientizando a los desarrolladores de las prácticas responsables de programación.

Capítulo 7

Organización de los sistemas operativos

La complejidad del tema de los sistemas operativos requiere que se haga de una forma modular. En este texto no se busca enseñar cómo se usa un determinado sistema operativo, ni siquiera comparar el uso de uno con otro (fuera de hacerlo con fines de explicar diferentes implementaciones).

En el nivel que se estudiará, un sistema operativo es más bien un gran programa, que ejecuta otros programas y les provee un conjunto de interfaces para que puedan aprovechar los recursos de cómputo. Hay dos formas primarias de organización *interna* del sistema operativo: los sistemas monolíticos y los sistemas microkernel. Y si bien no se puede marcar una línea clara a rajatabla que indique en qué clasificiación cae cada sistema, no es difícil encontrar líneas bases.

Monolíticos La mayor parte de los sistemas operativos históricamente han sido *monolíticos*: esto significa que hay un sólo *proceso privilegiado* (justamente el sistema operativo) que opera en modo supervisor, y dentro del cual se encuentran todas las rutinas para las diversas tareas que realiza el sistema operativo.

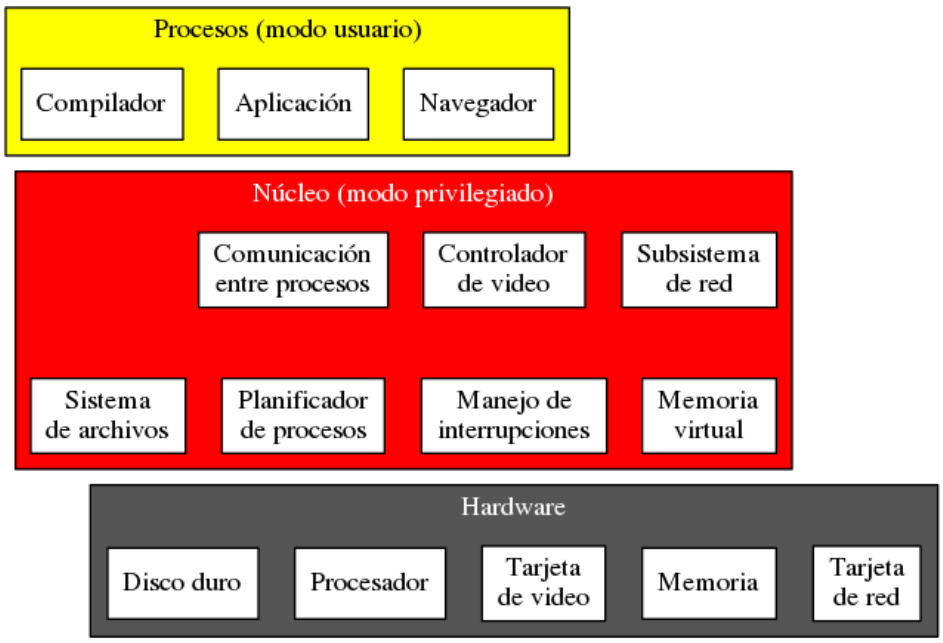


Figura 7.1: Esquematización de los componentes en un sistema monolítico.

Microkernel El núcleo del sistema operativo se mantiene en el mínimo posible de funcionalidad, descargando en *procesos especiales sin privilegios* las tareas que implementan el acceso a dispositivos y las diversas políticas de uso del sistema.

La principal ventaja de diseñar un sistema siguiendo un esquema monolítico es la simplificación de una gran cantidad de mecanismos de comunicación, que lleva a una mayor velocidad de ejecución (al requerir

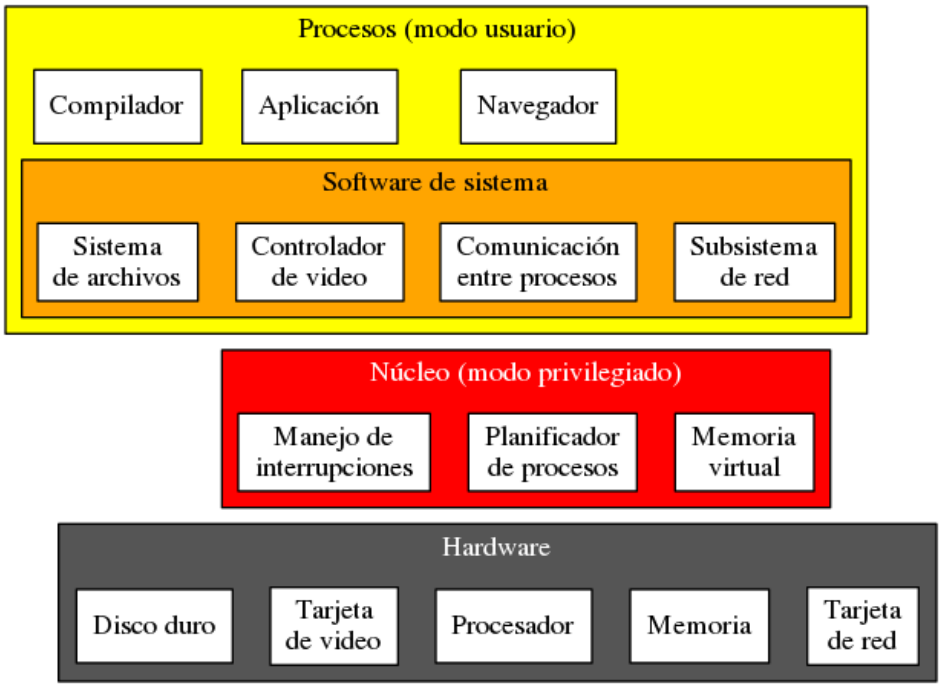


Figura 7.2: Esquematización de los componentes en un sistema microkernel.

menos cambios de contexto para cualquier operación realizada). Además, al manejarse la comunicación directa como paso de estructuras en memoria, el mayor acoplamiento permite más flexibilidad al adecuarse para nuevos requisitos (al no tener que modificar no sólo al núcleo y a los procesos especiales, sino también la interfaz pública entre ellos).

Por otro lado, los sistemas microkernel siguen esquemas lógicos más limpios, permiten implementaciones más elegantes y facilitan la comprensión por separado de cada una de sus piezas. Pueden *auto-repararse* con mayor facilidad, dado que en caso de fallar uno de los componentes (por más que parezca ser de muy bajo nivel), el núcleo puede reiniciarlo o incluso reemplazarlo.

Sistemas con concepciones híbridas No se puede hablar de concepciones únicas ni de verdades absolutas. A lo largo del libro se verán ejemplos de *concepciones híbridas* en este sentido: sistemas que son mayormente monolíticos pero que manejan algunos procesos que parecerían centrales mediante de procesos de nivel usuario como los microkernel (por ejemplo, los sistemas de archivos en espacio de usuario, FUSE, en Linux).

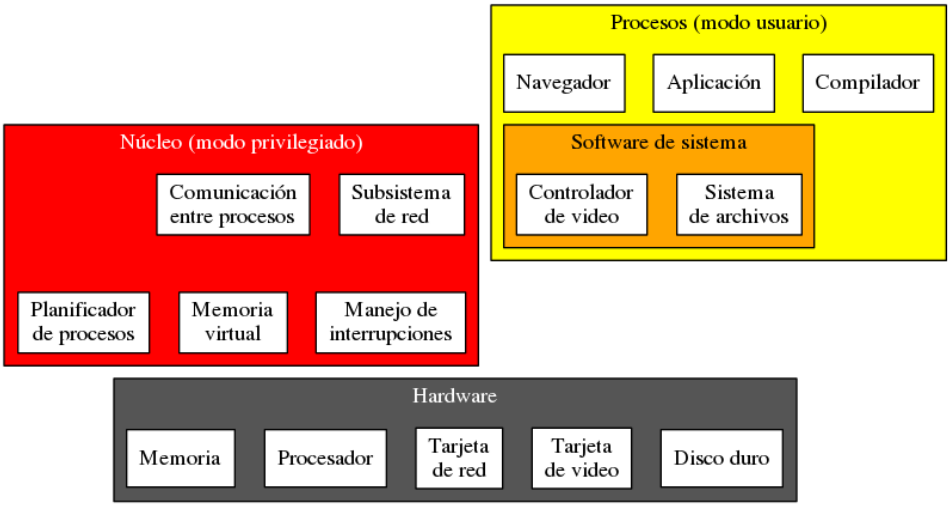


Figura 7.3: Esquematización de los componentes en un sistema híbrido.

Capítulo 8

Ejercicios

8.1. Preguntas de autoevaluación

1. ¿En qué se diferencia la multitarea apropiativa de la cooperativa? Para todas las opciones, léase: "A diferencia de la multitarea cooperativa, la apropiativa…" (Seleccione al menos una respuesta.)

   a) Es inmune a que un cálculo demasiado largo o un ciclo infinito dejen a la computadora efectivamente congelada.

   b) Es la más utilizada hoy en día.

   c) Ocurre sólo cuando el proceso hace una llamada al sistema.

   d) Se emplea principalmente en sistemas multiusuario.

   e) Requiere apoyo de hardware.

2. Un sistema operativo ofrece una serie de recursos o características principales, tanto a sus usuarios como a sus programadores. Estos pueden agruparse en *aislamiento*, *administración de recursos* y *abstracción*.

   De las siguientes afirmaciones, ¿cuál responde a cada uno de dichos conceptos, y cuál no corresponde a una función del sistema operativo?

   - Instrumentar políticas que repartan la atención del sistema de forma efectiva y acorde a las necesidades establecidas entre los diferentes procesos o usuarios.
   - Cada proceso y cada usuario no tendrán que preocuparse por otros que estén usando el mismo sistema; idealmente, su experiencia será la misma que si el sistema estuviera exclusivamente dedicado a su atención. Requiere que el sistema operativo cuente con ayuda del hardware.

- Presentar una interfaz consistente al usuario (puede ser gráfica o textual), eliminando las diferencias que provendrían de manejar distintos tipos de hardware.
- Los programadores no deben tener que preocuparse de los detalles del acceso a hardware, o de la configuración particular de una computadora. El programador debe poder enfocarse en resolver los problemas o necesidades particulares de sus usuarios.

3. Algunos dispositivos requieren de bloqueo para garantizar a un programa su acceso exclusivo. ¿Cuáles de los siguientes entrarían en ese supuesto?

   a) Teclado.
   b) Unidad de cinta.
   c) Discos.
   d) Impresora.

4. Un programa típicamente pasa por varias etapas en su ejecución, algunas de las cuales están limitadas por el procesador, mientras que las otras lo están por la entrada/salida. Los componentes del sistema que están ocupados en cada caso son distintos.

   ¿Qué tipo de sistemas nacieron para responder a esta necesidad?

5. Se presentó que los sistemas microkernel se basan en la simplificación de los mecanismos de comunicación y un esquema más claro de comunicación entre componentes. Sin embargo, los sistemas monolíticos siempre fueron más simples de implementar, razón por la cual muchos sistemas microkernel se han reducido a ejercicios académicos. Explique esta tensión.

6. De los sistemas operativos ampliamente utilizados que conozca, averigüe cuáles son microkernel y cuáles son monolíticos.

7. Los sistemas operativos empleados para dispositivos móviles son los mismos que los que utilizan las computadoras personales, sin embargo, hay áreas particulares, como la interfaz al usuario o el manejo de la energía, que son claramente distintos: ¿cómo puede verse la influencia en el sentido inverso? Esto es, ¿qué tanto ha influido la popularización de los dispositivos móviles en el camino de los sistemas operativos en general?

8.2. Lecturas relacionadas

- Fernando J. Corbató (2007). «ACM Turing Award Lectures». En: New York, NY, USA: ACM. Cap. On Building Systems That Will Fail. ISBN: 978-1-4503-1049-9. DOI: 10.1145/1283920.1283947. URL: http://doi.acm.org/10.1145/1283920.1283947
- R. Bjork (2000). URL: http://cs.gordon.edu/courses/cs322/lectures/history.html
- Michael Kerrisk (2013). *Making EPERM friendlier*. URL: http://lwn.net/Articles/532771/ explica algunas de las limitantes de la semántica POSIX como la falta de granularidad en el reporte de mensajes de error (EPERM) y errno global por hilo.
- Joel Spolsky (2003). *Biculturalism*. URL: http://www.joelonsoftware.com/articles/Biculturalism.html
- Jonathan Zittrain (2008). *The future of the Internet and how to stop it*. Yale University Press. URL: http://futureoftheinternet.org/

Parte II

Relación con el hardware

Capítulo 9

Introducción

Todos los sistemas de cómputo están compuestos por al menos una unidad de proceso junto con dispositivos que permiten ingresar datos (teclado, mouse, micrófono, etc.) y otros que permiten obtener resultados (pantalla, impresora, parlantes, etc.). Como se vio anteriormente, una de las funciones del sistema operativo es la de abstraer el hardware de la computadora y presentar al usuario una versión unificada y simplificada de los dispositivos. En este capítulo se verá la relación que mantiene el sistema operativo con el hardware, las funciones que cumplen y algunas abstracciones comunes utilizadas en sistemas operativos modernos.

Capítulo 10

Unidad de procesamiento

Es la parte fundamental de todo sistema de cómputo. Esta es la encargada de ejecutar tanto los programas del usuario como el sistema operativo en sí mismo. La funciones del sistema operativo respecto a la unidad de procesamiento son:

Inicialización Luego de ser cargado el sistema operativo debe realizar varias tareas de inicialización como habilitar las interrupciones de hardware y software (excepciones y trampas), configurar el sistema de memoria virtual (paginación, segmentación), etcétera.